OPLOGS-99: Best Practice Summary

> **Series:** OPLOGS — OpenPipeline Logs | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 04/25/2026

Definitive best practice settings for OpenPipeline log processing. Each entry specifies the exact configuration — no hedging, no options.

Table of Contents

1. Pipeline Configuration
2. Bucket Design & Retention
3. Data Masking
4. DQL Query Patterns
5. Metric Extraction
6. Cost Optimization
7. Security & Access Control
8. Monitoring & Alerting

---

1. Pipeline Configuration

Practice	Recommended Setting/Value	Priority
Process data at ingestion, not query time	Configure parsing, enrichment, masking, and routing as OpenPipeline processors	Critical
Pipeline stage order	Filter → Parse → Enrich → Mask → Extract → Route (masking BEFORE storage)	Critical

Practice	Recommended Setting/Value	Priority
First-match routing rule order	Most specific rules first, default catch-all last	Critical
Drop health check logs at ingestion	Drop processor: `contains(content, "health") AND loglevel == "INFO"`	Recommended
Drop DEBUG logs in production	Drop processor: `loglevel == "DEBUG"` or sample at 10%	Recommended
Parse NONE-level logs	DQL processor with matcher `loglevel == "NONE"` : parse content for level string	Recommended
Add computed environment attribute	`fieldsAdd environment = if(contains(k8s.namespace.name, "prod"), "production", else: "development")`	Recommended
Confirm 100% pipeline coverage	`countIf(isNotNull(dt.openpipeline.pipelines))` must equal total log count	Critical
Compare hourly volume before/after migration	`makeTimeseries count(), by: {dt.openpipeline.source}, interval:1h` — volume must be consistent	Critical

2. Bucket Design & Retention

Practice	Recommended Setting/Value	Priority
Create purpose-driven buckets	Minimum: `default_logs` , `debug_logs` , `error_logs` , `audit_logs` , `security_logs`	Critical
Bucket naming convention	`<purpose>_logs` or `<environment>_<purpose>_logs`	Recommended

Practice	Recommended Setting/Value	Priority
DEBUG/TRACE log retention	7 days	Critical
Standard INFO log retention	35 days	Recommended
ERROR log retention	90 days	Recommended
Audit log retention	365 days (730 for regulated industries)	Critical
Security log retention	180 days	Recommended
Verify retention compliance	`summarize earliest = min(timestamp), latest = max(timestamp), by:{dt.system.bucket}` — audit logs must show 365+ days	Critical

3. Data Masking

Practice	Recommended Setting/Value	Priority
Apply masking BEFORE storage	Masking processors execute first in pipeline — data never stored unmasked	Critical
Mask email addresses	Pattern: `[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}` → `[EMAIL-MASKED]`	Critical
Mask credit card numbers	Pattern: `\b(?:\d[ -]*?){13,16}\b` → `[CC-MASKED]`	Critical
Mask SSNs	Pattern: `\b\d{3}-\d{2}-\d{4}\b` → `[SSN-MASKED]`	Critical
Mask API keys and tokens	Pattern: `(api_key|apikey|key)=[A-Za-z0-9]{20,}` → `$1=[KEY-MASKED]`	Critical
Mask JWT tokens	Search for `eyJ` prefix, mask full token value	Critical
Mask passwords	Pattern: `password=`, `passwd=`, `pwd=` in content	Critical

Practice	Recommended Setting/Value	Priority
Use hash masking for correlation fields	"Hash value" processor (not "Mask value") when correlation is needed	Recommended
Run sensitive data discovery weekly	Search for `@`, `key=`, `token=`, `password`, `eyJ`, `card` patterns	Recommended
Run monthly PII exposure audit	Track `email_exposure_pct`, `key_exposure_pct`, `password_exposure_pct` — all must trend toward 0%	Critical

4. DQL Query Patterns

Practice	Recommended Setting/Value	Priority
Always specify time range	`fetch logs, from:-1h` — never bare `fetch logs`	Critical
Filter early, sort/limit last	`filter` immediately after `fetch`; `sort` and `limit` as final commands	Critical
Target specific buckets	`fetch logs, bucket:"error_logs", from:-7d`	Critical
Use `==` for exact matches	`filter field == "value"` — reserve `~` for wildcards only	Recommended
Always alias aggregations	`summarize count = count()` then `sort count desc` — anonymous aggregations cannot be referenced	Critical
Use `isNull()` / `isNotNull()`	Never `== null` — DQL uses three-valued logic	Critical
Use `in()` with curly-brace arrays	`in(status, {"error", "warning"})` — never SQL-style `IN ('a', 'b')`	Critical
Use `coalesce()` for fallbacks	`coalesce(loglevel, status, "UNKNOWN")` for null handling	Recommended

Practice	Recommended Setting/Value	Priority
Use `entityName()` for display names	`entityName(dt.entity.host, type:"dt.entity.host")` — avoids lookup overhead	Recommended
Filter `isNotNull` before aggregating optional fields	Prevents null group-by values in aggregations	Recommended
Use `samplingRatio` for exploration	`fetch logs, from:-1d, samplingRatio:10` then multiply back	Recommended
Cap query cost with `scanLimitGBytes`	`fetch logs, scanLimitGBytes:100`	Optional

5. Metric Extraction

Practice	Recommended Setting/Value	Priority
Extract dimensional metrics at ingestion	Create `log.request.count`, `log.response.duration`, `log.error.count` with service/method/status dimensions	Recommended
Minimum metric dimensions	`k8s.namespace.name`, `k8s.workload.name`, `loglevel`	Recommended
Use reverse-DNS for business event types	`com.example.payment.completed` format	Optional
Generate business events from log patterns	Order completion, user signup, deployment events	Optional

6. Cost Optimization

Practice	Recommended Setting/Value	Priority
Calculate droppable log percentage weekly	Health check + heartbeat + debug as % of total — target dropping 10-30%	Recommended
Measure storage by log level daily	`fieldsAdd content_bytes = stringLength(content)` then `summarize sum(content_bytes)/1048576.0, by:{loglevel}`	Recommended
Track bucket volume distribution weekly	Query all buckets showing total logs, unique sources, debug %, error % per bucket	Recommended

7. Security & Access Control

Practice	Recommended Setting/Value	Priority
Bucket-level IAM policies	`"permissions": ["storage:logs:read"], "conditions": [{"bucket": ["audit_logs"]}]`	Critical
Default bucket: all-authenticated read	Grant `storage:logs:read` on `default_logs` to all authenticated users	Recommended
Audit bucket: restricted access	`storage:logs:read` on `audit_logs` exclusively to audit team group	Critical
Classify IPs as internal vs external	Parse with DPL `IPADDR`, classify `10.*` / `192.168.*` / `172.*` as RFC1918, rest as EXTERNAL	Recommended
Monitor failed auth attempts	`(contains(content, "failed") OR contains(content, "denied")) AND (contains(content, "login") OR contains(content, "auth"))` at 30-min intervals	Critical

8. Monitoring & Alerting

Practice	Recommended Setting/Value	Priority
Error rate alert threshold	>5% per entity over 15-minute window, minimum 100 records	Recommended
Error count threshold	>10 per namespace per 15 minutes	Recommended
Pod crash loop detection	Filter for startup/initializing content per pod, alert when >3 in 1 hour	Recommended
Security error trends	`makeTimeseries count(), interval:30m` filtered on unauthorized/forbidden/denied	Recommended
Weekly bucket health check	Total logs, unique sources, debug %, error % per bucket over 7 days	Recommended

---